

DOCUMENT DE LA CONCEPTION DÉTAILLÉE

Projet : Déquantification de Données

Algorithme de Reconstruction par Élagage d'Arbre Binaire

Institution	Université Paris Cité
Langage	C (C99/C11)
Bibliothèque graphique	SDL2
Référence scientifique	Halalchi, Mahé, Jaïdane — ICASSP 2009
Source des données	helios2.mi.parisdescartes.fr (~mahe/dequantif/)

Rédacteurs : Chakib Mohamed Tahar Mahleb - Adem Rzaigui.

A- Contexte et problématique

Introduction

Ce document présente la conception détaillée du projet de déquantification de données développé en langage C à l'Université Paris Cité. L'objectif principal est de retrouver une série temporelle originale à partir de sa version sous-quantifiée, en exploitant l'histogramme statistique des couples de valeurs successives.

Ce document s'organise en sept parties. La première décrit le contexte scientifique du projet. La deuxième précise le format des données fournies. La troisième expose le principe de sous-quantification. La quatrième présente la structure de données retenue pour l'histogramme. La cinquième détaille l'algorithme de déquantification. La sixième décrit l'interface graphique. La septième présente l'architecture modulaire du projet.

1. Contexte Scientifique

1.1 La Quantification et ses Limites

La quantification réduit le nombre de bits utilisés pour représenter une valeur numérique. Cette réduction introduit une perte de précision. Elle intervient dans de nombreux domaines : compression multimédia (image, audio, vidéo), stockage embarqué et optimisation des réseaux de neurones.

On parle de sous-quantification lorsqu'on supprime délibérément des bits de poids faible. Cette opération est irréversible. Une valeur sous-quantifiée $xQ(n)$ peut correspondre à plusieurs valeurs originales distinctes.

1.2 Objectif : la Déquantification

Les travaux de Halalchi, Mahé et Jaïdane (ICASSP 2009, référence [1]) montrent qu'une reconstruction de l'histogramme original est possible sous certaines conditions statistiques. Pour une série temporelle, l'histogramme des couples de valeurs successives pourrait porter suffisamment d'information pour guider la reconstruction.

L'histogramme P est défini comme suit : $P(i, j)$ représente le nombre de fois où la paire $(x(n-1), x(n))$ prend les valeurs correspondant aux indices i et j . Cet histogramme capture les dépendances entre échantillons consécutifs.

1.3 Données Fournies

Le professeur Gaël Mahé met à disposition des fichiers binaires (.dat) sur son serveur universitaire. Chaque fichier contient une série temporelle. Le nom du fichier indique le nombre d'échantillons : `x10.dat` contient 10 échantillons, `x10000.dat` en contient 10 000.

Ces fichiers ne contiennent pas d'histogramme précalculé. Le programme doit le construire lui-même à partir de la série lue.

Point confirmé par le professeur

Les fichiers .dat stockent uniquement la série temporelle originale x_n , en entiers signés sur 16 bits (type short en C, format little-endian). L'histogramme P est calculé par le programme, à partir de cette série, avant de lancer la déquantification.

2. Format des Données

2.1 Nature des Fichiers

Les fichiers .dat sont des fichiers binaires purs. Ils ne contiennent ni en-tête ni séparateur. Les valeurs brutes des échantillons y sont encodées de façon consécutive. L'ouverture avec un éditeur de texte produit des symboles illisibles : c'est le comportement attendu pour un fichier binaire.

2.2 Encodage des Échantillons

Chaque échantillon est un entier signé sur 16 bits. En C, ce type correspond à short. Un short occupe 2 octets en mémoire. Les valeurs possibles s'étendent de -32 768 à +32 767. Pour obtenir le nombre d'échantillons d'un fichier, il suffit de diviser sa taille en octets par 2.

Propriété	Détail
Type C correspondant	short (entier signé 16 bits)
Taille par échantillon	2 octets
Plage de valeurs	-32 768 à +32 767
Encodage	Little-endian (architecture x86/x64)
Mode d'ouverture	Binaire : "rb"
Fonction de lecture	fread() — la fonction fscanf() est inadaptée aux fichiers binaires

3. Sous-Quantification

3.1 Principe

La sous-quantification sur k bits supprime les k bits de poids faible d'une valeur entière. Pour k = 1 (le cas de notre projet), le résultat est le nombre pair inférieur ou égal.

L'opération s'effectue par un double décalage binaire. Un décalage de k positions vers la droite supprime les k bits de poids faible. Un décalage de k positions vers la gauche remet la valeur à son échelle d'origine, avec des zéros aux positions des bits perdus.

3.2 Illustration pour $k = 1$

Valeur originale x	Valeur sous-quantifiée x_Q
5	4 (5 arrondi au pair inférieur)
-3	-4 (les négatifs sont arrondis vers le bas)
12	12 (valeur déjà paire, inchangée)
7	6
-1	-2

B. Description algorithmique du projet

0. Introduction

Ceci n'est qu'un pseudo code qui est indépendant du langage de programmation choisi et le programme finale sera plus ou moins modifiée en fonction de ce dernier.

1. Création de la série sous-quantifiée $x_Q(n)$

1.1. Principe

Lire chaque valeur x depuis le fichier binaire, calculer x_Q , puis écrire directement dans un nouveau fichier

1.2. Pseudo-code principal

```
Fonction quantize ()  
    //Input.file (x.dat) → output.file (Xq.dat)  
    debut  
        ouvrir input_file en mode "rb"  
        ouvrir output_file en mode "wb"  
  
        tant que on peut lire une valeur  $x$  depuis input_file:  
            lire  $x$  (type short)  
            // si  $x$  est pair on garde la même valeur ,  $x-1$  sinon  
            //ceci peut changer dans l'implémentation de l'algorithme en C  
            si  $x \geq 0$ :  
                 $x_Q = (x / 2) * 2$   
            sinon:  
                 $x_Q = ((x - 1) / 2) * 2$   
            fin si  
            écrire  $x_Q$  dans output_file  
        fin tantque  
        fermer input_file  
        fermer output_file  
    fin
```

2. Création de l'histogramme

$$P_{x_Q}$$

2.1. Signature de la fonction :

```
fonction build_Pq(filename):  
    //Entrée : filename : chemin vers le fichier binaire contenant la  
    série  
    //sortie : T : tableau de listes de couples (Vpere, Vfils, count)
```

2.2 Initialisation de la table de hachage

```
// Créer le tableau T de taille SIZE  
pour i de 0 à SIZE-1:  
    T[i] = vide  
Fin pour
```

Au départ, toutes les cases sont vides

La valeur de SIZE = 10007 (pour mieux répartir les couples)

2.3 Ouverture du fichier

```
fichier = ouvrir(filename, "rb")  
  
si fichier est vide:  
    afficher "Erreur ouverture"  
    retourner NULL  
fin si
```

2.4 Lecture de la première valeur

```
si lire(fichier, Vpere) échoue:
    fermer fichier et retourner T
fin si
```

2.5 Boucle principale : lire les couples

```
tant que lire(fichier, Vfils) réussit:
    // Calcul de l'indice avec le hash
    indice = hash(Vpere, Vfils)

    // Chercher le couple dans T[indice]
    trouvé = faux
    pour chaque noeud dans T[indice]:
        si noeud.Vpere == Vpere et noeud.Vfils == Vfils:
            noeud.count += 1
            trouvé = vrai
    Fin si
fin pour
// Si pas trouvé → créer un nouveau couple
si trouvé == faux:
    nouveau = (Vpere, Vfils, count=1)
    ajouter nouveau au début de T[indice]

// Avancer : le fils devient le nouveau père
Vpere = Vfils
fin tantque
Fermer fichier
```

2.6 Fonction hash

```
fonction hash(Vpere, Vfils) → retourne indice  
début  
    Ck = 2 654 435 769 // constante de Knuth  
    indice = Vpere * Ck + (Vfils mod SIZE)  
    retourner indice  
fin
```

2.7 Remarque : Pourquoi utiliser une table de hachage ?

Matrice Classique

Un tableau bidimensionnel classique pour P devrait couvrir toutes les valeurs possibles d'un short signé dans chaque dimension, soit **65 536** valeurs par axe. Le tableau comporterait plus de 4 milliards de cases. Avec 2 octets par case, cela représente environ 8 gigaoctets de mémoire vive.

Pour une série de 10 000 échantillons, au plus 9 999 couples distincts existent. L'écrasante majorité des cases resterait vide. Une structure creuse est indispensable.

Table de Hachage

Une table de hachage est un tableau de taille fixe. Chaque case contient une liste chaînée de couples $(x(n-1), x(n))$ et de leurs compteurs d'occurrences. Seuls les couples réellement présents dans la série occupent de la mémoire.

Une fonction de hachage transforme chaque couple en un indice entre 0 et la taille de la table moins 1. Deux couples différents peuvent produire le même indice : c'est une collision. Les couples en collision sont chaînés dans la même liste. La consultation parcourt la liste jusqu'au couple exact.

Choix de la Taille et de la Fonction de Hachage

La taille de la table est fixée à 10 007, un nombre premier proche de 10 000. L'usage d'un nombre premier garantit une meilleure répartition des indices et réduit les collisions. Pour 10 000 échantillons, on obtient environ un couple par case en moyenne.

La fonction de hachage combine les deux valeurs du couple par multiplication avec la constante de Knuth (2 654 435 769), issue de la théorie des nombres. Elle produit une bonne dispersion des bits. Le résultat est réduit modulo la taille de la table.

Comparaison Mémoire

Approche	Mémoire requise (10 000 échantillons)
Matrice 2D classique	Environ 8 Go — dépasse les capacités d'un PC standard
Table de hachage	Environ 140 Ko — 100 000 fois plus économique

Comparaison de la Complexité Temporelle

Matrice 2D : l'accès aux données est direct et s'effectue en temps constant $O(1)$ par simple indexation de tableau. Cette structure offre une rapidité théorique maximale mais reste inutilisable à cause de son coût mémoire de 8 Go.

Table de hachage : grâce à la fonction de Knuth, l'accès aux compteurs reste en $O(1)$ en moyenne malgré l'utilisation de listes chaînées.

3. Création de l'histogramme

 P_x

-Grâce au théorème de **Widrow**, on peut conclure que l'histogramme P_x peut être déduit à partir de l'histogramme P_{x_Q} sous certaines conditions.

4. Construction et élagage de l'arbre

4.0. Principe général

On construit un arbre binaire à partir d'une racine vide.

Cette racine possède deux fils possibles correspondant aux deux valeurs possibles de $xQ[0]$.

À chaque niveau :

- On génère deux candidats ($xQ[n]$ et $xQ[n] + 1$)
- On vérifie avec P_x si la transition est valide
- Sinon la branche est coupée (élagage)
→ Une branche valide = une feuille atteignant la profondeur N

On affiche à chaque instant : Le nombre de branches valides (feuilles actuelles).

4.1. Fonction principale

Explication :

Initialise une racine vide et crée les deux premières branches possibles.

4.1.1 La structure Noeud :

```
structure Noeud:
    valeur           // valeur x(n)
    Px               // histogramme
    pere             // pointeur vers le père
    fils_gauche      // pointeur vers fils (valeur paire)
    fils_droit       // pointeur vers fils (valeur impaire)
fin structure
```

Remarque 01 : Pour **simplifier** la présentation dans ce pseudo-code, nous considérons l'histogramme **Px** comme une **matrice 2D**.

4.1.2 . La fonction build_tree

```
nb_noeuds_valides = 0 //variable Globale
fonction build_tree(Xq, Px)
// la série des Val quantifiée Xq & l'histogramme Px → l'arbre
début
    racine = nouveau Noeud
    racine.valeur = NULL
    racine.pere = NULL
    racine.Px = Px
    branches_valides = 0
    // Initialisation avec Xq[0]
    candidats = { Xq[0], Xq[0] + 1 }
    pour chaque valeur dans candidats:
        fils = creer_fils(racine, valeur)
        construire(fils, 1)
    fin pour
```

4.2. Fonction ‘construire’ récursive :

Explication :

Construit les branches récursivement et applique l'élagage.
Une branche devient valide uniquement si elle atteint la fin.

```
fonction construire(noeud, n)
// noeud courant & profondeur act → Rien
Début
// mis à jour de l'affichage sdl

    update_display(noeud)

    si n == N: // Si on arrive à la fin → branche valide
        nb_branches_valides = nb_branches_valides + 1
        afficher(nb_branches_valides)
        retourner // sortie de la fonction
    fin si

    candidats = {Xq[n], Xq[n] + 1}
    nb_fils = 0
    pour chaque valeur dans candidats:
```

4.2.2 Création d'un fils

Explication :

Crée un nouveau nœud, met à jour l'histogramme et l'attache à son père.

```
fonction creer_fils(pere, valeur)
//nœud pere & valeur du fils → noeud
début
    fils = nouveau Noeud
    fils.valeur = valeur
    fils.pere = pere
    // Mise à jour histogramme
    pere.Px[pere.valeur][valeur] -= 1
    fils.Px = pere.Px
    // Optimisation mémoire
    pere.Px = NULL
    // Attachement
    si valeur est paire:
        pere.fils_gauche = fils
    sinon:
```

4.2.3 Suppression (élagage)

Explication :

Supprime un nœud sans fils et **remonte si nécessaire**. (remonter lorsque le père n'a pas de fils valide , dans ce cas le père sera supprimer .

```
fonction supprimer_noeud(noeud) // nœud a supprimer → Rien
début
    pere = noeud.pere
    si pere != NULL:
        si pere.fils_gauche == noeud:
            pere.fils_gauche = NULL
        si pere.fils_droit == noeud:
            pere.fils_droit = NULL
    supprimer noeud
```

4.2.4. Vérification de transition

Explication :

Vérifie si un couple est autorisé par l'histogramme.

```
fonction existe_transition(Px, i, j) //histogramme & valeur act &val condidate  
→ Boolean  
début  
    retourner (Px[i][j] > 0)  
fin
```

4.3 Conclusion

À chaque rang n , pour chaque branche active, on connaît la valeur $x(n-1)$ déjà reconstruite. On examine les deux candidats pour $x(n)$. Pour chacun, on consulte P pour vérifier si le couple $(x(n-1), x(n))$ a un compteur strictement positif.

Un couple valide entraîne la décrémentation de son compteur dans P et la création d'un enfant dans l'arbre. Un couple invalide entraîne le rejet de la branche. Un nœud qui ne possède aucun enfant valide est élagué uniquement si le rang courant est strictement inférieur à $N-1$. L'élagage remonte récursivement jusqu'au premier ancêtre ayant encore un enfant valide.

Si le nœud atteint le rang $N-1$, il constitue une feuille représentant une série candidate complète.

Propriété clé de l'algorithme

L'efficacité de la déquantification dépend des dépendances statistiques de la série originale. Des dépendances fortes entre échantillons consécutifs enrichissent P, accélèrent l'élagage et favorisent la convergence vers une solution unique.

Dans la partie suivante du **conception détaillée**, nous orientons légèrement notre approche vers le **langage C**, car la partie graphique est plus complexe à représenter en pseudo-code. Cela permet de mieux illustrer les structures et les algorithmes sans perdre en clarté.

5. Interface Graphique

5.1 Exigence du Projet

Le cahier des charges impose un mode graphique pour visualiser la construction et l'élagage de l'arbre binaire rang par rang. Cette visualisation permet d'observer, en temps réel, quelles branches survivent et lesquelles sont élaguées.

5.2 Choix de SDL2

La bibliothèque SDL2 (Simple DirectMedia Layer, référence [3]) a été retenue pour l'interface graphique. SDL2 est multiplateforme et écrite en C, ce qui la rend compatible avec le reste du projet. Elle permet de créer une fenêtre, de tracer des formes géométriques et du texte, et de gérer les événements clavier et souris sans recourir à un framework lourd.

5.3 Contenu de la Visualisation

La fenêtre affiche l'arbre binaire de façon dynamique. Chaque nœud est représenté par un cercle portant la valeur reconstruite à ce rang. Les branches valides relient les nœuds par des traits. Les branches élaguées sont distinguées visuellement par une couleur différente ou une suppression progressive. L'utilisateur observe le rétrécissement de l'arbre jusqu'aux feuilles finales.

La progression de l'algorithme est visible rang par rang. À chaque rang n , les nouveaux nœuds apparaissent, les branches invalides disparaissent et l'arbre se met à jour. Le nombre de solutions restantes est affiché en temps réel.

5.4 Fonctions du Module display

Le module display est découpé en plusieurs fonctions précises, chacune assurant une responsabilité unique.

init_display()

Rôle : initialise SDL2 et crée la fenêtre principale du programme.

Fonctionnement : appelle **SDL_Init()** pour activer le sous-système vidéo de SDL2. Crée ensuite la fenêtre avec **SDL_CreateWindow()** en définissant son titre, ses dimensions et ses options d'affichage. Crée le renderer SDL2 avec **SDL_CreateRenderer()** qui servira à dessiner tous les éléments graphiques. En cas d'échec de l'une de ces étapes, affiche un message d'erreur et quitte proprement.

Appelée depuis : main.c, une seule fois au démarrage, avant tout dessin.

draw_node(SDL_Renderer* renderer, int x, int y, short value, int color)

Rôle : dessine un nœud de l'arbre à une position donnée dans la fenêtre.

Fonctionnement : trace un cercle rempli à la position (x, y) avec la couleur spécifiée. Inscrit la valeur du nœud en texte au centre du cercle à l'aide de **SDL_RenderDrawPoint()** ou d'une bibliothèque de texte comme **SDL_ttf**. La couleur distingue les nœuds actifs (bleu), les nœuds élagués (rouge) et les feuilles finales (vert).

Appelée depuis : **draw_tree()**, de façon récursive pour chaque nœud de l'arbre.

draw_edge(SDL_Renderer* renderer, int x1, int y1, int x2, int y2)

Rôle : dessine un trait reliant un nœud parent à un nœud enfant.

Fonctionnement : utilise **SDL_RenderDrawLine()** pour tracer une ligne entre les coordonnées (x1, y1) du parent et (x2, y2) de l'enfant. La couleur de la ligne reflète l'état de la branche : active ou élaguée.

Appelée depuis : **draw_tree()**, avant de dessiner chaque enfant.

draw_tree(SDL_Renderer* renderer, TreeNode* node, int x, int y, int spread, int depth)

Rôle : dessine récursivement l'intégralité de l'arbre binaire depuis sa racine.

Fonctionnement : reçoit le nœud courant, sa position (x, y) dans la fenêtre, l'écartement horizontal **spread** entre branches et la profondeur actuelle **depth**. Dessine d'abord le nœud courant via **draw_node()**. Si le nœud a un enfant gauche (branche paire), trace le trait vers lui et appelle **draw_tree()** récursivement avec **x - spread** et **y + espacement vertical**. Fait de même pour l'enfant droit (branche impaire) avec **x + spread**. L'écartement **spread** est réduit à chaque niveau pour éviter le chevauchement des nœuds.

Appelée depuis : **update_display()**, à chaque mise à jour de l'affichage.

update_display(SDL_Renderer* renderer, TreeNode* root, int current_rank)

Rôle : efface l'écran et redessine l'arbre dans son état courant au rang n.

Fonctionnement : appelle `SDL_RenderClear()` pour effacer le contenu précédent. Affiche le rang courant en texte en haut de la fenêtre. Appelle `draw_tree()` depuis la racine pour redessiner l'arbre complet. Appelle `SDL_RenderPresent()` pour envoyer le résultat à l'écran. Une pause courte (`SDL_Delay()`) permet à l'utilisateur de voir chaque étape.

Appelée depuis : la boucle principale de `dequantize()` dans `tree.c`, à chaque rang n.

handle_events()

Rôle : gère les événements utilisateur pendant l'exécution.

Fonctionnement : appelle `SDL_PollEvent()` en boucle pour récupérer les événements en attente. Si l'utilisateur ferme la fenêtre (`SDL_QUIT`) ou appuie sur la touche Échap, déclenche l'arrêt propre du programme. Permet également de mettre en pause l'animation avec la barre espace.

Appelée depuis : `update_display()`, à chaque mise à jour de l'affichage.

close_display(SDL_Renderer* renderer, SDL_Window* window)

Rôle : libère toutes les ressources SDL2 et ferme la fenêtre.

Fonctionnement : appelle `SDL_DestroyRenderer()` pour libérer le renderer. Appelle `SDL_DestroyWindow()` pour fermer la fenêtre. Appelle `SDL_Quit()` pour arrêter le sous-système SDL2. Cette fonction est toujours appelée en fin d'exécution, même en cas d'erreur.

Appelée depuis : `main.c`, juste avant le `return 0` final.

5.5 Tableau Récapitulatif des Fonctions de display

Fonction	Rôle	Appelée depuis
<code>init_display()</code>	Initialise SDL2 et crée la fenêtre	<code>main.c</code> (une seule fois)
<code>draw_node()</code>	Dessine un cercle avec la valeur du nœud	<code>draw_tree()</code> (récursif)
<code>draw_edge()</code>	Trace un trait parent → enfant	<code>draw_tree()</code> (récursif)
<code>draw_tree()</code>	Dessine récursivement l'arbre complet	<code>update_display()</code>
<code>update_display()</code>	Efface et redessine à chaque rang n	<code>dequantize()</code> dans <code>tree.c</code>
<code>handle_events()</code>	Gère fermeture, pause, touche Échap	<code>update_display()</code>
<code>close_display()</code>	Libère SDL2 et ferme la fenêtre	<code>main.c</code> (fin d'exécution)

6. Architecture du Projet

6.1 Arborescence Complète des Fichiers

Le projet est organisé selon la structure suivante. Chaque module regroupe un fichier d'en-tête (.h) et un fichier d'implémentation (.c). Les fichiers de données sont regroupés dans un dossier séparé.

```

projet/
├── main.c           ← point d'entrée, orchestre tout
├── series.h         ← interface : read_series, quantize_series
├── series.c         ← implémentation lecture et quantification
├── histogram.h      ← interface : hash_table, Node, fonctions P
├── histogram.c      ← implémentation table de hachage
├── tree.h           ← interface : TreeNode, fonctions arbre
├── tree.c           ← implémentation arbre binaire et élagage
├── display.h        ← interface : fonctions SDL2
├── display.c        ← implémentation interface graphique
└── data/           ← dossier des fichiers de données
    ├── x10.dat      ← 10 échantillons (données de test)
    ├── x100.dat     ← 100 échantillons
    └── x10000.dat   ← 10 000 échantillons

```

6.2 Liens entre les Fichiers

Chaque flèche indique qu'un fichier inclut (#include) le fichier pointé et utilise ses fonctions ou types.

```

main.c
├──▶ series.h      (utilise read_series, quantize_series)
├──▶ histogram.h   (utilise build_histogram, free_histogram)
├──▶ tree.h        (utilise dequantize, print_solutions, free_tree)
└──▶ display.h     (utilise init_display, close_display)

tree.c
├──▶ tree.h        (ses propres prototypes et TreeNode)
├──▶ histogram.h   (utilise get_count, decrement)
└──▶ display.h     (appelle update_display à chaque rang)

display.c
├──▶ display.h     (ses propres prototypes)
└──▶ tree.h        (accès à TreeNode pour le dessin)

histogram.c → histogram.h (ses propres prototypes et Node)

```

```
series.c    → series.h    (ses propres prototypes)
```

7. Programme Principale :

Début

Appeler **quantize ()**

//Création de la série sous-quantifiée $x_Q(n)$ à partir de la série $x(n)$

Appeler **build_Pq(filename):**

//Création de l'histogramme

P_{x_Q}
Déduire de l'histogramme P_x à partir de

Appeler **build_tree()**

//Construction et élagage de l'arbre

Afficher les statistiques : comparer les séries résultantes avec la série originale x et afficher le pourcentage de réussite.

fin

Le programme suit cinq étapes successives. Il lit la série originale $x(n)$ depuis le fichier .dat passé en argument. Il calcule la série sous-quantifiée $x_Q(n)$ par décalage binaire. Il construit l'histogramme P des couples successifs. Il exécute l'algorithme d'élagage sur l'arbre binaire et en parallèle affiche le résultat et le nombre de solutions via SDL2.

8. Glossaire

Terme	Définition
Quantification	Réduction du nombre de bits utilisés pour représenter une valeur numérique, entraînant une perte de précision.

Sous-quantification	Cas particulier de quantification consistant à supprimer les k bits de poids faible d'une valeur entière.
xQ(n)	Valeur de la série sous-quantifiée au rang n, obtenue en effaçant les k bits de poids faible de x(n).
Histogramme P	Matrice P(i, j) comptant le nombre d'occurrences du couple (x(n-1), x(n)) dans la série originale.
Table de hachage	Structure de données stockant uniquement les couples existants de P, à l'aide d'une fonction de hachage.
Fonction de hachage	Fonction transformant un couple de valeurs en un indice entier dans le tableau de la table de hachage.
Collision	Situation où deux couples différents produisent le même indice de hachage. Résolue par liste chaînée.
Arbre binaire	Structure représentant l'ensemble des séries candidates à la déquantification, avec deux branches par rang.
Élagage	Suppression des branches de l'arbre dont le couple (x(n-1), x(n)) est absent ou épuisé dans P.
Feuille	Nœud terminal de l'arbre représentant une série déquantifiée candidate complète.
short	Type entier signé sur 16 bits en C, correspondant au format de stockage des échantillons dans les fichiers .dat.
little-endian	Convention d'encodage où l'octet de poids faible est stocké en premier en mémoire.
SDL2	Bibliothèque multiplateforme en C pour la création de fenêtres et le rendu graphique.
fread()	Fonction C de lecture binaire d'un fichier, lisant un nombre précis d'octets sans interprétation.

9. Références

- [1] H. Halalchi, G. Mahé, M. Jaïdane, « Revisiting quantization theorem through audiowatermarking », Proc. ICASSP 2009, avril 2009, pp. 3361-3364.
- [2] D. E. Knuth, The Art of Computer Programming, Vol. 3 : Sorting and Searching, 2e édition, Addison-Wesley, 1998. (Constante de hachage multiplicatif de Fibonacci.)
- [3] SDL2 — Simple DirectMedia Layer, documentation officielle : <https://wiki.libsdl.org/SDL2/>
- [4] Site des données du projet : <https://helios2.mi.parisdescartes.fr/~mahe/dequantif/>

MARS, 2026